

✓ SafeGirl — Intent Classifier Development

DistilBERT Intent Classification

This notebook documents the development, training, validation, and evaluation of the SafeGirl intent classification component.

The classifier is designed to categorize adolescent sexual and reproductive health queries into the following intent categories:

1. Contraception
2. STI
3. Pregnancy
4. General

The GBV category is handled separately by the SafeGirl SafetyNet because GBV-related and crisis-related queries require a dedicated safety/referral pathway rather than ordinary informational generation.

Research Objective

The objective of this stage is to develop and evaluate a transformer-based intent classifier using multilingual DistilBERT.

The classifier will be evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Per-class performance
- Confusion matrix

An independent test set will be kept separate from training and validation and will only be used for final model evaluation.

Notebook Principle

This notebook follows a reproducible development sequence:

Environment → Data → Quality Checks → Split → Tokenization → Training → Validation → Error Analysis → Independent Testing → Model Artefacts

1. Reproducibility and Data Controls

The classifier development process uses explicit controls to reduce data leakage and improve reproducibility.

Dataset Controls

- Original seed queries are retained as the source dataset.
- Generated paraphrases are reviewed before model development.
- Train/validation splitting is performed at the `(seed_id)` level.
- Paraphrases derived from the same seed must not cross the training and validation partitions.
- The independent test set is kept separate from training and validation.
- The independent test set is used only for final evaluation.

Model Controls

- Random seed: `(42)`
- Base model: `(distilbert-base-multilingual-cased)`
- GBV is handled through the SafetyNet rather than the transformer classifier.

Evaluation Principle

The validation set supports model development and model selection.

The independent test set remains unseen until final evaluation.

2. Computational Environment

Model development is conducted in Google Colab using a CUDA-enabled GPU environment.

The computational environment is established before dataset processing or model training to ensure that the training procedure is reproducible and that the transformer model can make use of GPU acceleration.

The main libraries used for this stage are:

- PyTorch — model computation and GPU acceleration
- Transformers — DistilBERT implementation and training
- Datasets — dataset handling
- Scikit-learn — evaluation metrics
- Pandas — tabular data processing
- Accelerate — efficient transformer training

```
# SafeGirl — Install required machine-learning dependencies

!pip install -q \
    transformers==5.16.1 \
    datasets==4.0.0 \
    scikit-learn==1.6.1 \
    pandas==2.2.3 \
    accelerate==1.14.0
```

3. Environment Verification

Before proceeding to dataset processing, the installed software versions and GPU availability are verified.

Training will proceed only after confirming that CUDA is available and that the expected GPU is accessible.

```
# SafeGirl — Verify training environment

import torch
import transformers
import datasets
import sklearn
import pandas
import accelerate

print("=== SafeGirl Training Environment ===")
print(f"PyTorch:      {torch.__version__}")
print(f"Transformers:  {transformers.__version__}")
print(f"Datasets:      {datasets.__version__}")
print(f"Scikit-learn:   {sklearn.__version__}")
print(f"Pandas:         {pandas.__version__}")
print(f"Accelerate:     {accelerate.__version__}")
print()

cuda_available = torch.cuda.is_available()
print(f"CUDA available: {cuda_available}")

if cuda_available:
    print(f"CUDA version:  {torch.version.cuda}")
    print(f"GPU:            {torch.cuda.get_device_name(0)}")
else:
    raise RuntimeError(
        "CUDA GPU is not available. Stop before continuing with training."
    )
```

```
=== SafeGirl Training Environment ===
PyTorch:      2.11.0+cu128
Transformers:  5.16.1
Datasets:      4.0.0
Scikit-learn:  1.6.1
Pandas:        2.2.3
Accelerate:    1.14.0

CUDA available: True
CUDA version:   12.8
GPU:            Tesla T4
```

4. SafeGirl Dataset

The classifier is trained using reviewed query paraphrases derived from the original SafeGirl seed queries.

The dataset is organized into:

- `dataset/seeds/` — original seed queries and seed-level split assignments
- `dataset/reviewed/` — manually reviewed paraphrased queries
- `dataset/test/` — independent test data reserved for final evaluation

Only the reviewed development data will be used during the training and validation stages.

The independent test dataset will remain untouched until final evaluation.

```
from google.colab import drive
drive.mount("/content/drive")
```

```
Mounted at /content/drive
```

Dataset Path Configuration

The following paths point to the SafeGirl project dataset within Google Drive.

The paths should be adjusted to match the location of the SafeGirl project in the mounted Drive.

```
from pathlib import Path

drive_root = Path("/content/drive/MyDrive")

# Search for directories named "dataset"
dataset_matches = list(drive_root.rglob("safegirl_dataset"))

print("Dataset directories found:")
for path in dataset_matches[:20]:
    print(path)
```

```
Dataset directories found:
/content/drive/MyDrive/safegirl_dataset
```

5. Dataset Path Configuration

The SafeGirl dataset is stored in Google Drive and accessed through the mounted Colab runtime.

The dataset root contains the original seed queries, reviewed paraphrases, and the independent test-set directory.

Only the reviewed development data will be accessed during training and validation. The independent test set remains reserved for final evaluation.

```
from pathlib import Path

# SafeGirl dataset root
DATA_DIR = Path("/content/drive/MyDrive/safegirl_dataset")

SEEDS_DIR = DATA_DIR / "seeds"
REVIEWED_DIR = DATA_DIR / "reviewed"
TEST_DIR = DATA_DIR / "test"

print("=== SafeGirl Dataset Structure ===")
print(f"Dataset root: {DATA_DIR}")
print(f"Seeds directory exists: {SEEDS_DIR.exists()}")
print(f"Reviewed directory exists: {REVIEWED_DIR.exists()}")
print(f"Test directory exists: {TEST_DIR.exists()}")
```

```
=== SafeGirl Dataset Structure ===
Dataset root: /content/drive/MyDrive/safegirl_dataset
Seeds directory exists: True
Reviewed directory exists: True
Test directory exists: False
```

```
print("\n=== Dataset Contents ===")
```

```

for directory in [SEEDS_DIR, REVIEWED_DIR, TEST_DIR]:
    print(f"\n{directory}:")

    if directory.exists():
        files = sorted(directory.iterdir())

        if files:
            for file in files:
                print(f" - {file.name}")
        else:
            print(" (empty)")
    else:
        print(" DIRECTORY NOT FOUND")

```

```

=== Dataset Contents ===

```

```

/content/drive/MyDrive/safegirl_dataset/seeds:
- seed_split_assignment.csv
- seeds.csv

/content/drive/MyDrive/safegirl_dataset/reviewed:
- contraception_paraphrases.csv
- general_paraphrases (1).csv
- pregnancy_paraphrases.csv
- sti_paraphrases (1).csv

/content/drive/MyDrive/safegirl_dataset/test:
DIRECTORY NOT FOUND

```

6. Reviewed Dataset Inspection

The reviewed dataset is inspected before model training to establish its actual size, structure, labels, and data quality.

This step verifies that the reviewed paraphrases contain the information required for supervised intent classification.

No model training is performed during this stage.

```

import pandas as pd

# Load all reviewed CSV files
reviewed_files = sorted(REVIEWED_DIR.glob("*.csv"))

reviewed_data = {}

for file in reviewed_files:
    df = pd.read_csv(file)
    reviewed_data[file.name] = df

    print(f"\n{'=' * 70}")
    print(f"FILE: {file.name}")
    print(f"{'=' * 70}")
    print(f"Rows: {len(df)}")
    print(f"Columns: {list(df.columns)}")
    print()
    print(df.head(3).to_string(index=False))

```

```

=====
FILE: contraception_paraphrases.csv
=====
Rows: 56
Columns: ['seed_id', 'paraphrase_id', 'class', 'text']

```

seed_id	paraphrase_id	class	text
SEED-CONTRA-01	SEED-CONTRA-01-P1	contraception	If I get the pill will my parents find out?
SEED-CONTRA-01	SEED-CONTRA-01-P2	contraception	Do I have to tell my mom or dad before getting birth control?
SEED-CONTRA-01	SEED-CONTRA-01-P3	contraception	Is there a way to get on birth control that my parents won't know about?

```

=====
FILE: general_paraphrases (1).csv
=====
Rows: 40
Columns: ['seed_id', 'paraphrase_id', 'class', 'text']

```

seed_id	paraphrase_id	class	text
SEED-GEN-01	SEED-GEN-01-P01	general	What does "youth-friendly" mean when a clinic offers health services?
SEED-GEN-01	SEED-GEN-01-P02	general	What makes a health clinic "youth-friendly"?

SEED-GEN-01 SEED-GEN-01-P03 general What should I expect from a youth-friendly health clinic?

=====

FILE: pregnancy_paraphrases.csv

=====

Rows: 60

Columns: ['seed_id', 'paraphrase_id', 'class', 'text']

seed_id	paraphrase_id	class	text
SEED-PREG-01	SEED-PREG-01-P1	pregnancy	How many times do I need to go for antenatal checkups?
SEED-PREG-01	SEED-PREG-01-P2	pregnancy	What's the recommended number of antenatal visits?
SEED-PREG-01	SEED-PREG-01-P3	pregnancy	How often should I attend antenatal care during my pregnancy?

=====

FILE: sti_paraphrases (1).csv

=====

Rows: 64

Columns: ['seed_id', 'paraphrase_id', 'class', 'text']

seed_id	paraphrase_id	class	text
SEED-STI-01	SEED-STI-01-P01	sti	How can I tell whether I might have an STI?
SEED-STI-01	SEED-STI-01-P02	sti	What are the signs that I could have an STI?
SEED-STI-01	SEED-STI-01-P03	sti	How would I know if I had an STI?